Практическая работа 7
Разработка приложения для построения схем метро

Для начала просмотрим код:

```
import flet as ft
import uuid
import math

class Station:
    def __init__(self, x, y, name, line_color):
        self.id = str(uuid.uuid4())
        self.x = x
        self.y = y
        self.name = name
        self.line_color = line_color

class Edge:
    def __init__(self, station1, station2, distance, line_color):
        self.id = str(uuid.uuid4())
        self.station1 = station1
        self.station2 = station2
        self.distance = distance
        self.line_color = line_color

class MetroApp:
    def __init__(self):
        self.stations = []
        self.edges = []
        self.selected_station = None
        self.selected_edge = None
        self.connecting_from = None
        self.line_colors = ['red', 'blue', 'green', 'orange', 'purple', 'yellow', 'brown', 'pink']
        self.current_color_index = 0

    def get_next_color(self):
        color = self.line_colors[self.current_color_index]
        self.current_color_index = (self.current_color_index + 1) % len(self.line_colors)
        return color

def main(page: ft.Page):
    page.title = 'Редактор схемы метро'

    app = MetroApp()
```

```
    name_input = ft.TextField(label='Название станции', width=200)
    distance_input = ft.TextField(label='Расстояние', width=100,
keyboard_type=ft.KeyboardType.NUMBER)

    status_text = ft.Text('Введите название станции и кликните на поле для
добавления', size=16)

    canvas_stack = ft.Stack()

    def draw():
        canvas_stack.controls.clear()


        for edge in app.edges:
            is_selected = app.selected_edge and app.selected_edge.id == edge.id
            color = 'white' if is_selected else edge.line_color

            x1, y1 = edge.station1.x, edge.station1.y
            x2, y2 = edge.station2.x, edge.station2.y

            length = int(((x2 - x1) ** 2 + (y2 - y1) ** 2) ** 0.5)
            steps = max(length // 3, 1)
            for i in range(steps + 1):
                t = i / steps if steps > 0 else 0
                px = x1 + (x2 - x1) * t
                py = y1 + (y2 - y1) * t
                canvas_stack.controls.append(
                    ft.Container(
                        width=4,
                        height=4,
                        left=px - 2,
                        top=py - 2,
                        bgcolor=color,
                        border_radius=2,
                    )
                )

            mid_x = (x1 + x2) / 2
            mid_y = (y1 + y2) / 2
            canvas_stack.controls.append(
                ft.Container(
                    content=ft.Text(str(edge.distance), size=10, color='white'),
                    left=mid_x - 10,
                    top=mid_y - 10,
                    bgcolor='black',
```

```
                padding=2,
            )
        )

    for station in app.stations:
        is_selected = app.selected_station and app.selected_station.id == station.id
        radius = 18 if is_selected else 14

        canvas_stack.controls.append(
            ft.Container(
                left=station.x - radius,
                top=station.y - radius,
                width=radius * 2,
                height=radius * 2,
                border_radius=radius,
                bgcolor=station.line_color,
                content=ft.Text(station.name[0], color='white', size=12,
weight=ft.FontWeight.BOLD, text_align=ft.TextAlign.CENTER),
            )
        )

        canvas_stack.controls.append(
            ft.Text(
                station.name,
                size=11,
                left=station.x + 20,
                top=station.y - 7,
                color='white' if is_selected else 'black',
            )
        )

    page.update()

click_pos = {'x': 400, 'y': 300}

def on_tap_down(e: ft.TapEvent):
    """Handle tap down event with coordinates."""
    click_pos['x'] = e.local_x
    click_pos['y'] = e.local_y
    process_click()

def process_click():
    x = click_pos['x']
    y = click_pos['y']
```

```
        for station in app.stations:
            if abs(station.x - x) < 20 and abs(station.y - y) < 20:
                if app.connecting_from:
                    if app.connecting_from.id != station.id:
                        dist = int(distance_input.value) if distance_input.value else 1
                        edge = Edge(app.connecting_from, station, dist,
app.connecting_from.line_color)
                        app.edges.append(edge)
                    app.connecting_from = None
                    status_text.value = 'Режим добавления'
                else:
                    app.selected_station = station
                    app.selected_edge = None
                draw()
                return

        if name_input.value:
            color = app.get_next_color()
            station = Station(x, y, name_input.value, color)
            app.stations.append(station)
            app.selected_station = station
            name_input.value = ''
            draw()

    def connect_mode(e):
        if app.selected_station:
            app.connecting_from = app.selected_station
            status_text.value = f'Выберите станцию для соединения с
{app.selected_station.name}'
            page.update()

    def edit_distance(e):
        if app.selected_edge and distance_input.value:
            app.selected_edge.distance = int(distance_input.value)
            draw()

    def delete_selected(e):
        if app.selected_station:
            app.edges = [ed for ed in app.edges if ed.station1.id != app.selected_station.id
and ed.station2.id != app.selected_station.id]
            app.stations = [s for s in app.stations if s.id != app.selected_station.id]
            app.selected_station = None
        elif app.selected_edge:
            app.edges = [ed for ed in app.edges if ed.id != app.selected_edge.id]
            app.selected_edge = None
```

```
        draw()

    def clear_all(e):
        app.stations = []
        app.edges = []
        app.selected_station = None
        app.selected_edge = None
        app.connecting_from = None
        draw()

    canvas_content = ft.Container(
        content=canvas_stack,
        width=800,
        height=550,
        bgcolor='#444444',
        border=ft.border.all(2, 'gray'),
    )

    canvas = ft.GestureDetector(
        on_tap_down=on_tap_down,
        content=canvas_content,
    )

    page.add(
        ft.Column([
            ft.Text('Редактор схемы метро', size=24, weight=ft.FontWeight.BOLD),
            ft.Row([
                name_input,
                distance_input,
                ft.Button('Соединить', on_click=connect_mode),
            ]),
            ft.Row([
                ft.Button('Удалить', on_click=delete_selected),
                ft.Button('Очистить', on_click=clear_all),
            ]),
            status_text,
            canvas,
        ])
    )

    draw()

if __name__ == "__main__":
    ft.app(target=main)
```

Теперь посмотрим на работу приложения на рисунке 1:

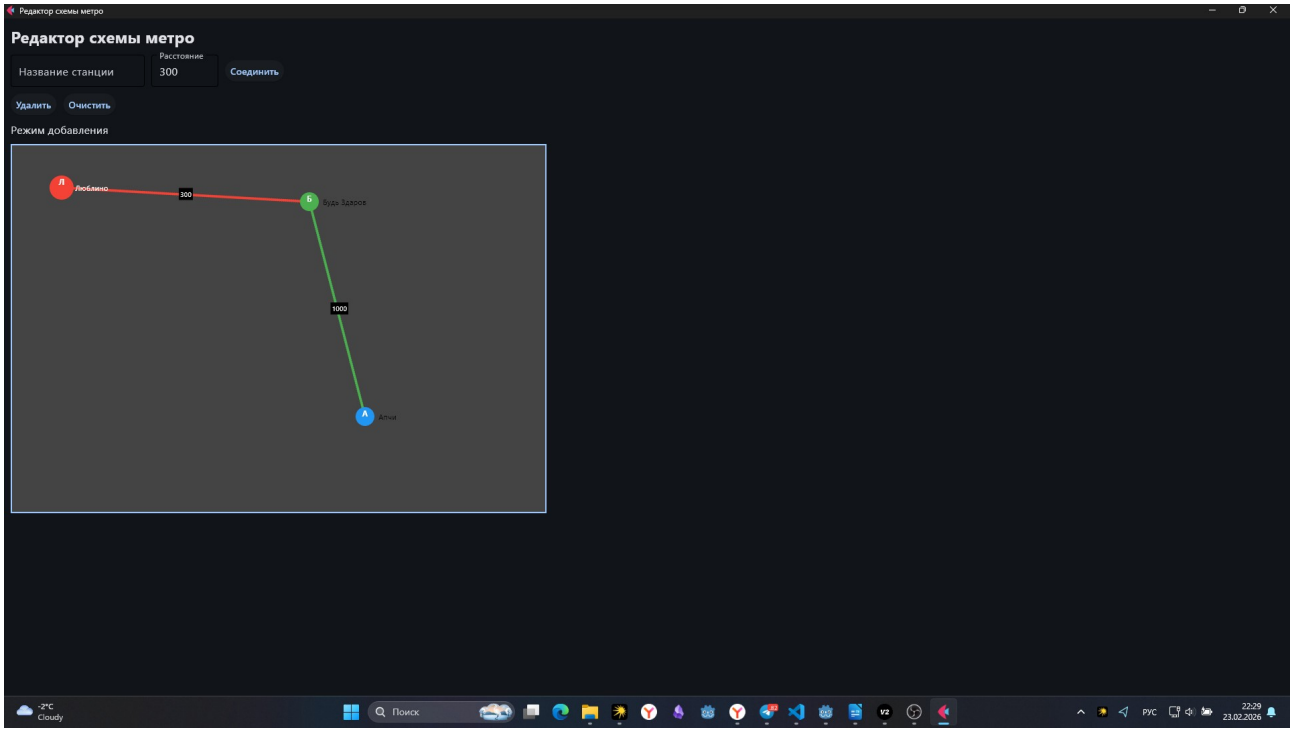

Рисунок 1 — Демонстрация работы